Ishimwe Pacis Hanyurwimfura
Student ID: 3787234

Lab5: MPI Matrix–Vector Multiplication and Benchmarking Report

The values provided being the average after multiple runs

Matrix Size	P=1 (seconds)	P=2(seconds)	P=4(seconds)	P=8(seconds)
1000	0.006674	0.005477	0.006440	0.011964
5000	0.184513	0.212355	0.099438	0.102113
10000	0.876237	0.461924	0.411224	0.405909
20000	7.946844	2.311501	2.036975	1.843162

Speed Up table

Computation S = T_serial / T_parallel

Matrix Size	P=1	P=2	P=4	P=8
1000	1	1.219	1.036	0.558
5000	1	0.869	1.856	1.807
10000	1	1.897	2.131	2.159
20000	1	3.438	3.901	4.312

Questions for discussions

1. Does the program achieve linear speedup?

   No Looking at the Speed Up Table for example for P=8 we can see with the matrix size being 10000 it was 2.159 and when it is doubled(20000) it is 4.312 which is good but not linear and also we can see for the small

matrices it's being slower as process increase due to the communication.

2. If not, what factors limit scalability?
    a. MPI communication overhead: The program is scattering the matrix, broadcasting the vector and then gather the result, this communication takes time and becomes significant when the computation is small.
    b. Synchronization cost: We are using the barrier on the timed section and from this the faster process can be delayed by slow ones.
    c. Memory bandwidth and memory access cost: The Matrix-vector multiplication needs to read many values from memory and when more processes run at the same time, they can compete for access to memory, and some processes spend time waiting for data instead of computing so adding more processes does not always give proportional speedup.

3. Why does increasing N sometimes improve speedup?

    As N increases, speedup often improves because the program spends more time doing the actual computation and less time being affected by MPI communication overhead. For small matrices, the costs of scattering the matrix, broadcasting the vector, gathering the results, and synchronizing processes can take up a large part of the total runtime differently for larger matrices as each process has more rows to compute so the communication overhead becomes a smaller part of the work making parallel execution more effective.

4. Why can extremely large values of N cause crashes or segmentation faults?

    Extremely large values of N can cause crashes or segmentation faults because the program stores the matrix as a dense N×N array which requires a very large amount of memory. Since each matrix element is a double, the memory needed grows quickly as N increases. The root process must allocate the entire matrix, so for very large sizes the

system may run out of available memory, which can lead to allocation failure, a crash, or a segmentation fault.